agentic-ai-azure-week03-04-agent-framework

Weeks 3-4 - Microsoft Agent Framework

Agentic AI on Azure - Enterprise Master Class | Handout (slides + notes)

Notes

Welcome. This deck covers 'agentic-ai-azure-week03-04-agent-framework' - Weeks 3-4 - Microsoft Agent Framework. Frame the problem and why it matters, then walk the class through the learning goal, the enterprise use case, the architecture/flow, the key concepts, a live run in VS Code, and the architect's takeaways. The repo runs locally with no Azure, so demo it live.

Slide 2 - Learning goal

- Use memory, middleware and grounding tools
- Stream responses; define agents declaratively
- Apply governance without touching agent logic

Notes

[Learning goal] Walk through these talking points:
- Use memory, middleware and grounding tools
- Stream responses; define agents declaratively
- Apply governance without touching agent logic

Ask the class: which of these ideas is new to you?

Slide 3 - Enterprise use case

- Wealth Management Research Assistant
- Pulls holdings + market data, keeps session memory
- Redacts PII in; injects compliance disclaimer out

Notes

[Enterprise use case] Walk through these talking points:
- Wealth Management Research Assistant
- Pulls holdings + market data, keeps session memory
- Redacts PII in; injects compliance disclaimer out

Tip: relate this to a regulated scenario your students know.

Slide 4 - Architecture / flow

- Tools: get_holdings, get_market_data
- Middleware: PII redaction (in) + disclaimer (out)
- Memory: in-memory (mock) or durable Cosmos DB
- Partitioned by sessionId; survives restarts

Notes

[Architecture / flow] Walk through these talking points:
- Tools: get_holdings, get_market_data
- Middleware: PII redaction (in) + disclaimer (out)
- Memory: in-memory (mock) or durable Cosmos DB
- Partitioned by sessionId; survives restarts

Demo: trace a single request through these steps live.

Slide 5 - Key concepts

- Pluggable memory store (InMemory / Cosmos)
- Middleware as a governance seam
- Memory tiers: Redis vs. Cosmos vs. AI Search

Notes

[Key concepts] Walk through these talking points:
- Pluggable memory store (InMemory / Cosmos)
- Middleware as a governance seam
- Memory tiers: Redis vs. Cosmos vs. AI Search
Open the named file in the repo while you explain each point.

Slide 6 - Run it (VS Code - no Azure needed)

- git clone the repo, then open it in VS Code
- python -m venv .venv -> activate it
- pip install -r requirements.txt
- uvicorn app.main:app --reload
- Open /docs and call POST /api/v1/research
- Runs in MOCK mode - no Azure/keys needed

Notes

[Run it (VS Code - no Azure needed)] Walk through these talking points:
- git clone the repo, then open it in VS Code
- python -m venv .venv -> activate it
- pip install -r requirements.txt
- uvicorn app.main:app --reload
- Open /docs and call POST /api/v1/research
- Runs in MOCK mode - no Azure/keys needed
Pause for questions before moving on.

Slide 7 - Architect's takeaways

- Cost & consistency trade-offs across memory tiers
- Declarative (YAML) vs. code-first for CI/CD
- Enforce policy at the seam, not in business logic

Notes

[Architect's takeaways] Walk through these talking points:
- Cost & consistency trade-offs across memory tiers
- Declarative (YAML) vs. code-first for CI/CD
- Enforce policy at the seam, not in business logic
Discussion: when would you NOT choose this approach?